4-bit Array Multiplier from RTL to GDSII

SkyWater SKY130 · `sky130_fd_sc_hd` · open-source flow

Name	Roll number	Parts
Sale Sangameshwar	2024102017	A (RTL design and verification), B (logic synthesis)
Palla Rithik Reddy	2024102005	C (static timing analysis), D (physical design)

Repository: `github.com/rithikreddypalla/DVD_RTL_to_GDS` October 2026

Summary of results

Check	Result
RTL simulation, 256 vectors	**PASS**: `PASS: 256 vectors checked, 0 errors`
Fault-injection test	caught: `FAIL: 256 vectors checked, 95 errors`
Synthesis (Yosys)	61 cells, 756.976 µm^2, 16 flip-flops (52.89 % of area)
Gate-level simulation	**PASS**: `PASS: 256 vectors checked, 0 errors`
STA, typical corner (OpenSTA)	$T_{\min} = 2.787$ ns, $F_{\max} = 358.81$ MHz
Place and route (LibreLane)	CLOCK_PERIOD 8.0 ns, setup slack +0.362 ns in all corners
Signoff	DRC 0 (Magic and KLayout), LVS 0, hold slack > 0 in all corners
Post-route $F_{\max}$	130.92 MHz (all corners), 264.72 MHz (typical corner)

Item	Version / value
Environment	IIC-OSIC-TOOLS Docker image `hpretl/iic-osic-tools:2026.10`, WSL 2 on Windows 11
Simulation	Icarus Verilog 14.0 (devel), GTKWave
Synthesis	Yosys 0.70
Timing	OpenSTA 3.1.0, typical corner `tt_025C_1v80`
Physical design	LibreLane v3.1.0.dev3 (OpenROAD, Magic, KLayout, Netgen)
PDK	`sky130A`, standard cells `sky130_fd_sc_hd`

1 Part A: RTL design and verification

1.1 Design

`rtl/full_adder.v` contains two continuous assignments:

$$sum = a \oplus b \oplus c_{in}, \qquad c_{out} = ab + a\,c_{in} + b\,c_{in}.$$

`rtl/mult_array.v` (ports `clk`, `rst_n`, `a[3:0]`, `b[3:0]`, `p[7:0]`) registers the inputs (`a_r`, `b_r`, clock edge 1) and the product (`p`, edge 2) in one `always @(posedge clk or negedge rst_n)` block: 16 flip-flops, an asynchronous active-low reset and a latency of 2 clock cycles. Between the registers, 16 AND gates form $pp_i[j] = a_r[j] \wedge b_r[i]$ and 12 `full_adder` instances (`fa1_0` … `fa3_3`) are connected exactly as in the table below. The `*` operator is not used.

Stage	Inputs of full adder j	Outputs
Row 0	none (AND gates only)	$p[0] = pp_0[0]$; $r_0 = \{0, pp_0[3], pp_0[2], pp_0[1]\}$
Row 1 (`fa1_0`–`fa1_3`)	$pp_1[j]$, $r_0[j]$, $c_1[j]$, $c_1[0] = 0$	$p[1] = s_1[0]$; $r_1 = \{c_1[4], s_1[3], s_1[2], s_1[1]\}$
Row 2 (`fa2_0`–`fa2_3`)	$pp_2[j]$, $r_1[j]$, $c_2[j]$, $c_2[0] = 0$	$p[2] = s_2[0]$; $r_2 = \{c_2[4], s_2[3], s_2[2], s_2[1]\}$
Row 3 (`fa3_0`–`fa3_3`)	$pp_3[j]$, $r_2[j]$, $c_3[j]$, $c_3[0] = 0$	$p[3] = s_3[0]$; $p[7:4] = \{c_3[4], s_3[3], s_3[2], s_3[1]\}$

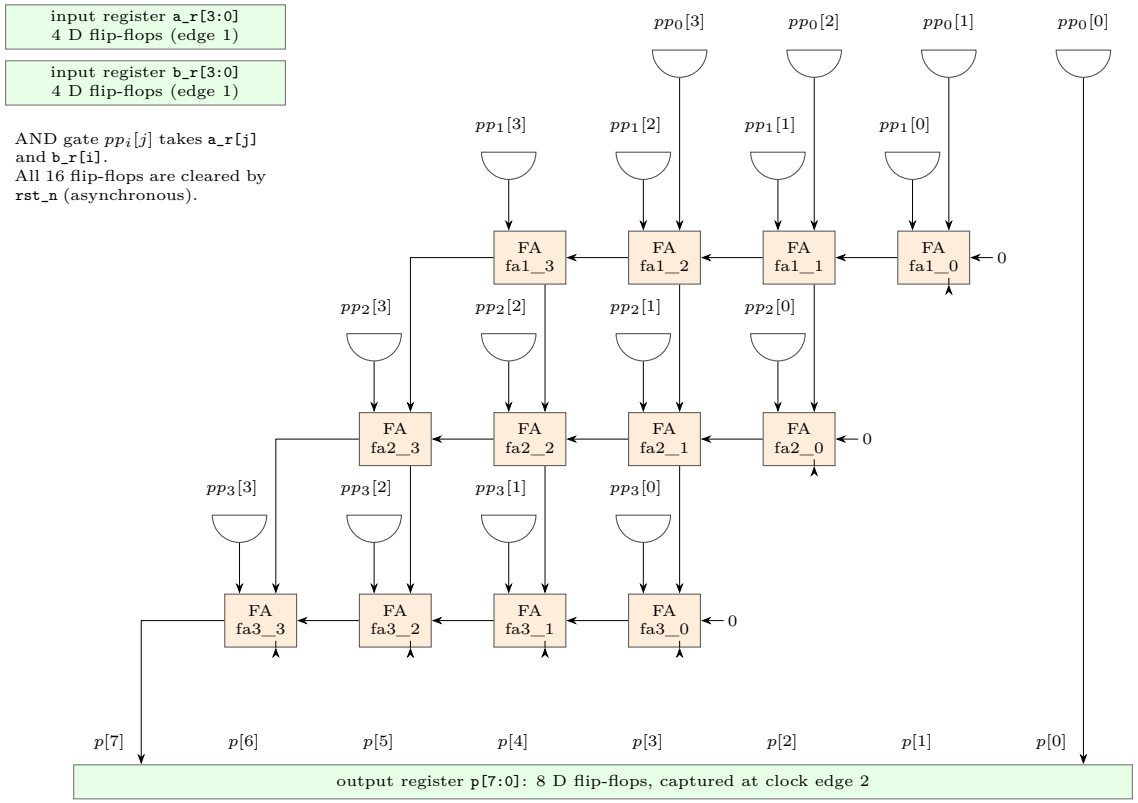


Figure 1: Block diagram of `mult_array`: 16 AND gates, 12 full adders in three ripple-carry rows, 8 input and 8 output flip-flops. Columns are aligned by bit weight, so every passed-down bit $r_{i-1}[j]$ is a vertical wire; carries ripple right to left inside each row.

1.2 Testbench and simulation

`tb/tb_mult.v` runs a 10 ns clock with `rst_n` low for the first 2 cycles (checking $p = 0$), applies a new (a, b) pair on every falling edge for all 256 combinations and compares `p` with `a * b` exactly 2 cycles later, so a design with the wrong latency fails; `!==` makes X or Z an error. It prints one line per mismatch, one summary line, and writes `mult.vcd`. The commands are listed in Appendix A.

Run	Summary line
Correct RTL	`PASS: 256 vectors checked, 0 errors`
Fault injected: `assign sum = a ^ b;`	`FAIL: 256 vectors checked, 95 errors`
Fault removed (restored)	`PASS: 256 vectors checked, 0 errors`

With the fault, the first mismatch is `a=3 b=3 expected p=9 got p=5` at 560 ns: the first vector in which a full adder sees a carry-in of 1, which is exactly the term the fault removes.

1.3 Hand-worked example

Roll number used: Sale Sangameshwar, 2024102017. The last digit is 7, so $A = 7 + 5 = 12$, and since $A \neq 7$, $B = 7$.

```
          1 1 0 0     A = 12
        x 0 1 1 1     B = 7
        ---------
          1 1 0 0     pp0 = A AND b0 (b0 = 1)
        1 1 0 0       pp1 = A AND b1 (b1 = 1), shifted 1
      1 1 0 0         pp2 = A AND b2 (b2 = 1), shifted 2
    0 0 0 0           pp3 = A AND b3 (b3 = 0), shifted 3
  ---------------
  0 1 0 1 0 1 0 0     P = 84
```

Traced through the array (bits written MSB first; full adders $j = 0..3$ listed as (sum, carry)):

Row	Inputs	Full adders $j = 0..3$	Outputs
0	$pp_0 = 1100$	none	$p[0] = 0$, $r_0 = 0110$
1	$pp_1 = 1100$, $r_0 = 0110$	(0,0) (1,0) (0,1) (0,1)	$p[1] = 0$, $r_1 = 1001$
2	$pp_2 = 1100$, $r_1 = 1001$	(1,0) (0,0) (1,0) (0,1)	$p[2] = 1$, $r_2 = 1010$
3	$pp_3 = 0000$, $r_2 = 1010$	(0,0) (1,0) (0,0) (1,0)	$p[3] = 0$, $p[7{:}4] = 0101$

The result is $p = 01010100_2 = 84$. In the waveform (vector $16 \times 12 + 7 = 199$, applied at $30 + 10 \times 199 = 2\,020$ ns) it is registered at edge 1 (2 025 ns) and $p = 84$ appears at edge 2 (2 035 ns).

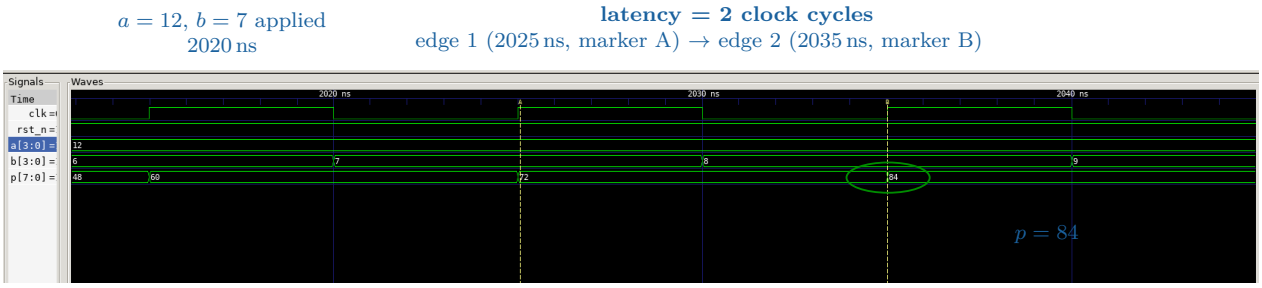


Figure 2: RTL waveform for $A = 12$, $B = 7$: the inputs change on the falling edge at 2 020 ns, are registered at edge 1 (marker A) and $p = 84$ is registered at edge 2 (marker B), a latency of 2 clock cycles.

2 Part B: Logic synthesis with Yosys

`synth/synth.tcl` reads the two RTL files and runs `synth -top mult_array -flatten`, `dfflibmap`, `abc` (both with the typical-corner Liberty file `sky130_fd_sc_hd__tt_025C_1v80.lib`), `opt_clean -purge`, `stat -liberty` (to `synth/mult_array_stat.rpt`) and `write_verilog -noattr` (to `synth/mult_array_netlist.v`); it exits with code 0 and only informational ABC warnings. The netlist is then simulated with the same testbench and the SKY130 cell models (command in Appendix A) and prints `PASS: 256 vectors checked, 0 errors`.

One `dfrtp_1` (25.024 µm^2) is a D flip-flop with an active-low asynchronous reset, matching `rst_n`. The passing gate-level simulation shows that synthesis preserved the behaviour, including the 2-cycle latency and the reset.

3 Part C: Static timing analysis with OpenSTA

`sta/sta.tcl` loads the same Liberty file, links `synth/mult_array_netlist.v`, defines `clk` with period T, sets a 1.0 ns input delay on every input except `clk` and a 1.0 ns output delay on every

Metric	Value
Total number of cells	61
Number of flip-flops	16 (`sky130_fd_sc_hd__dfrtp_1`)
Total cell area (μm^2)	756.976
Flip-flop area (μm^2)	$16 \times 25.024 = 400.384$
Flip-flop area (% of total)	$100 \times 400.384/756.976 = 52.89\,\%$
The 3 most-used cell types and their counts	`dfrtp_1` ×16, `nand2_1` ×7, then `and2_0` and `maj3_1` ×4 (tied)
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

Table 1: Part B results (`synth/mult_array_stat.rpt`).

Cell	Count	Area (μm^2)	Cell	Count	Area (μm^2)
`dfrtp_1`	16	400.384	`nand3_1`	2	10.010
`nand2_1`	7	26.275	`nand4_1`	2	12.512
`and2_0`	4	25.024	`nor2_1`	2	7.507
`maj3_1`	4	40.038	`xor2_1`	2	17.517
`a21oi_1`	3	15.014	`a21boi_0`	1	7.507
`xnor2_1`	3	26.275	`and3_1`	1	6.256
`xor3_1`	3	71.318	`and4_1`	1	8.758
`a22oi_1`	2	15.014	`o2111ai_1`	1	8.758
`clkinv_1`	2	7.507	`o21bai_1`	1	7.507
`lpflow_isobufsrc_1`	2	12.512	`o311ai_0`	1	8.758
			`xnor3_1`	1	22.522
Total: 61 cells, $756.976\,\mu m^2$			Logic only: $61 - 16 = 45$ cells		

Table 2: Full cell list from the stat report.

output, prints the worst setup slack and writes the critical path to `sta/mult_array_timing.rpt` (the seven steps of the assignment). T is 10 ns unless `STA_PERIOD` is set. At $T = 10$ ns the worst slack is 7.213 ns, so $T_{\min} = T - \mathrm{WNS} = 10 - 7.213 = 2.787$ ns and $F_{\max} = 1000/2.787 = 358.81$ MHz; at $T = 2.79$ ns the slack is +0.003 ns, which confirms $T_{\min}$.

Metric	Value
Worst slack at $T = 10$ ns (ns)	7.213
$T_{\min}$ (ns)	2.787 (2.79 rounded to 0.01 ns)
$F_{\max}$ (MHz)	358.81
Slack at $T = T_{\min} = 2.79$ ns (ns)	0.003
Critical path start point	`_91_` = input register `a_r[1]` (`dfrtp_1`, CLK)
Critical path end point	`_88_` = output register `p[6]` (`dfrtp_1`, D)
Number of cells on the critical path	10 (2 flip-flops + 8 logic cells)
Latency (ns) = $2 \times T_{\min}$	5.574
Throughput (million results per second)	358.81 (one result per cycle)

Table 3: Part C results.

The instance names are generated by Yosys; they were mapped to the RTL through their D and Q nets in `synth/mult_array_netlist.v` (`_91_`: D = `a[1]`, Q = `a_r[1]`; `_88_`: D = `fa3_3.sum`, Q = `p[6]`).

All 8 logic cells implement full-adder logic, with the partial-product AND gates merged in by ABC. The path crosses 7 full-adder stages (`fa1_1` → `fa2_0` → `fa2_1` → `fa2_2` → `fa3_1` → `fa3_2` →

#	Instance	Cell	Delay (ns)	Arrival (ns)	Role in the array
1	_91_	dfrtp_1	0.398	0.398	launch flip-flop a_r[1], CLK → Q
2	_37_	clkinv_1	0.073	0.471	inverts a_r[1] for the merged row-1 logic
3	_42_	o311ai_0	0.356	0.827	row 1: part of $s_1[1]$ (fa1_1 sum, fa1_0 carry folded in)
4	_45_	a21oi_1	0.153	0.980	row 2: $c_2[1] = pp_2[0] \cdot r_1[0]$ (fa2_0, half adder)
5	_51_	maj3_1	0.402	1.382	row 2: $c_2[2]$ (fa2_1 carry)
6	_58_	xnor3_1	0.370	1.752	row 2: $s_2[2] = r_2[1]$ (fa2_2 sum)
7	_61_	maj3_1	0.368	2.119	row 3: $c_3[2]$ (fa3_1 carry)
8	_63_	maj3_1	0.354	2.474	row 3: $c_3[3]$ (fa3_2 carry)
9	_64_	xor3_1	0.178	2.652	row 3: $s_3[3] = p[6]$ (fa3_3 sum)
10	_88_	dfrtp_1	setup 0.135	2.787	capture flip-flop p[6]

Table 4: Cells on the critical path, in order (sta/mult_array_timing.rpt, $T = 2.79$ ns).

fa3_3). The second path group in the report is the reset recovery check (rst_n → RESET_B, slack 2.018 ns), which does not limit $F_{\max}$.

4 Part D: Physical design with LibreLane

The starting clock period is $1.2 \times T_{\min} = 1.2 \times 2.787 = 3.344$ ns, rounded up to the next 0.5 ns: 3.5 ns. pnr/config.json holds exactly the required keys:

```
{
    "DESIGN_NAME": "mult_array",
    "VERILOG_FILES": ["dir::../rtl/full_adder.v", "dir::../rtl/mult_array.v"],
    "CLOCK_PORT": "clk",
    "CLOCK_PERIOD": 8.0,
    "FP_SIZING": "absolute",
    "DIE_AREA": [0, 0, 150, 150]
}
```

4.1 Runs

LibreLane signs off timing in nine corners (min/nom/max wire RC × tt_025C_1v80, ss_100C_1v60, ff_n40C_1v95); timing__setup__ws is the worst of them. The typical corner met timing in every run; only the slow corner (1.60 V, 100 °C) failed, so CLOCK_PERIOD was raised by 0.5 ns after every failing run.

From 6.5 ns on, the slack stopped improving by 0.5 ns per step: LibreLane re-synthesizes the RTL with an ABC script that depends on CLOCK_PERIOD, and the resizer placed two clkdlybuf4s25_1 buffers (wire19, max_cap18) on the critical path, costing about 1.24 ns in the ss corner. Runs 9 and 10 have the same netlist (ss data arrival 7.643 ns), so 8.0 ns is the first period that covers it.

4.2 Results

Hold slack is positive in every corner (worst +0.141 ns), there are no antenna violations, and the only remaining checker message is a max-slew warning in the ss corners, which is not a signoff error.

Run	Folder RUN_2026-10-23_…	CLOCK_PERIOD (ns)	Setup WS, all corners (ns)	Setup WS, nom_tt (ns)	DRC / LVS	Result
1	23-04-36	3.5	−2.546	0.431	0 / 0	setup fails (ss)
2	23-09-59	4.0	−2.027	0.929	0 / 0	setup fails (ss)
3	23-15-41	4.5	−1.527	1.429	0 / 0	setup fails (ss)
4	23-19-59	5.0	−1.027	1.929	0 / 0	setup fails (ss)
5	23-24-04	5.5	−0.527	2.429	0 / 0	setup fails (ss)
6	23-28-24	6.0	−0.037	2.911	0 / 0	setup fails (ss)
7	23-32-27	6.5	−0.205	3.111	0 / 0	setup fails (ss)
8	23-36-21	7.0	−0.116	3.526	0 / 0	setup fails (ss)
9	23-40-39	7.5	−0.138	3.722	0 / 0	setup fails (ss)
10	23-44-39	**8.0**	**+0.362**	4.222	0 / 0	**flow complete**

Table 5: Every LibreLane attempt (run folders under `pnr/runs/`). DRC is the Magic and KLayout count, both 0 in every run.

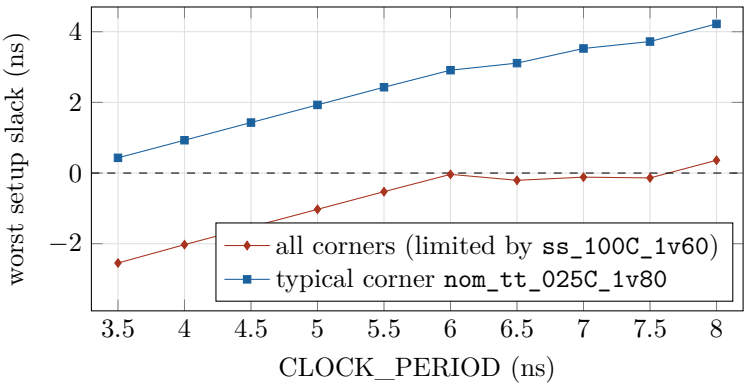


Figure 3: Setup slack against CLOCK_PERIOD for the ten runs. The flow passes when the red curve crosses 0.

Metric	Value
CLOCK_PERIOD used (ns)	8.0
Number of runs needed	10
Core area (μm^2)	17 759.5 (die area 22 500 = 150 × 150)
Cells after synthesis (Part B)	61
Cells after place and route	1 884 (`design__instance__count`)
Kinds of cells added	decap, fill, tap, port buffers, repair buffers, clock buffers (counts and purpose in Section 4.3)
Wirelength (µm)	2 518 (detailed route)
Setup slack (ns)	+0.362 (worst corner `max_ss_100C_1v60`); +4.222 in `nom_tt_025C_1v80`
Post-route F_{max} (MHz)	1000/(8.0 − 0.362) = 130.92 (all corners); 1000/(8.0 − 4.222) = 264.72 (typical corner)
F_{max} from Part C (MHz)	358.81 (typical corner, ideal clock, no wires)
DRC errors	0 (Magic 0, KLayout 0)
LVS errors	0

Table 6: Part D results (`pnr/metrics.json`).

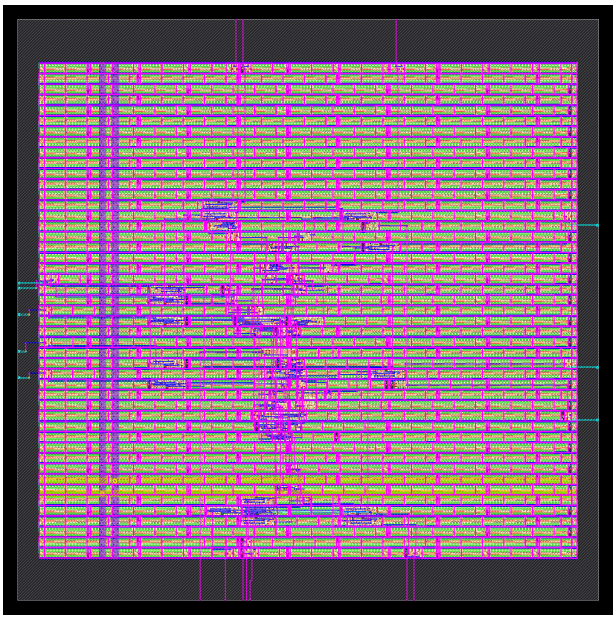
4.3 Cells added by the flow

1 884 − 61 = 1 823 cells were added. They were counted by cell name in the final DEF:

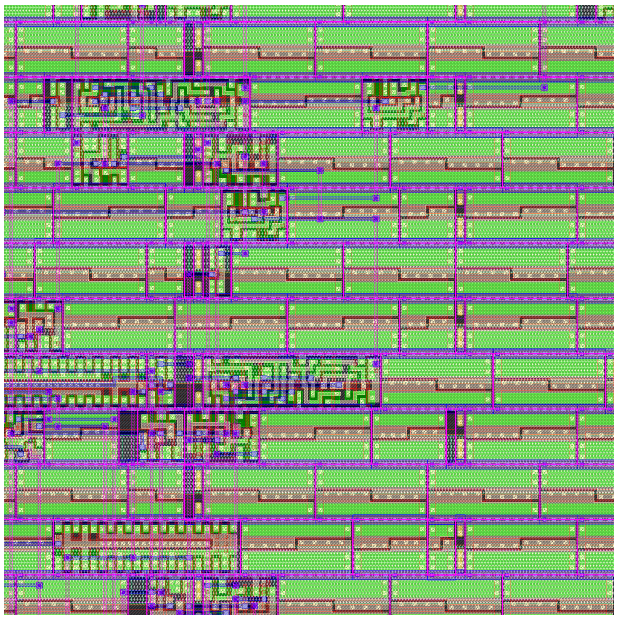
Kind	Cells	Count	Why it is needed
Logic (LibreLane synthesis)	`dfrtp_1` ×16 + 46 gates	62	the multiplier itself (Part B gave 61 with a different ABC script)
Decap	`decap_3/4/6/8/12`	1 299	on-chip capacitance between VPWR and VGND that steadies the supply when cells switch
Fill	`fill_1`, `fill_2`	254	close the gaps in the rows so wells, implants and power rails stay continuous
Tap	`tapvpwrvgnd_1`	245	tie the n-well to VPWR and the substrate to VGND at regular spacing (latch-up)
Port buffers	`buf_2` ×17 (`input1`–`input9`, `output10`–`output17`)	17	isolate the 9 input and 8 output pins from the core and drive the output load
Repair buffers	`clkdlybuf4s50_1` ×2, `clkdlybuf4s25_1` ×2	4	fix fanout, long-wire and max-capacitance violations
Clock tree	`clkbuf_16` ×3	3	clock tree synthesis: one root and two leaf buffers drive the 16 flip-flops with low skew

Most of the added cells are decap and fill, because the logic covers only 7.1 % of the 17 759.5 µm^2 core.

4.4 Layout



(a) Whole 150 µm × 150 µm die. Standard-cell rows fill the core with the met4/met5 power straps; pins sit on the die edges; the logic sits near the centre.



(b) 30 µm × 30 µm zoom at the centre of the core: standard-cell rows with met1 power rails, logic cells between fill and decap cells, and met1/met2 signal wires with vias.

Figure 4: Final layout `pnr/mult_array.gds` in KLayout (SKY130 layer colours).

5 Answers to the report questions

1. Logic cells after synthesis. Synthesis produced 61 cells, 16 of them flip-flops, so $61 - 16 = 45$ cells implement the logic, not $16 + 12 = 28$. With `synth -flatten` ABC maps the whole array as one Boolean network instead of 12 adder blocks. A full adder needs at least two library cells (for example `xor3_1` + `maj3_1`), but ABC also merges AND gates into compound cells (`a21oi`, `a22oi`, `o2111ai`, `nand3`/`nand4`) and simplifies the adders with a constant 0 input ($c_i[0] = 0$ in every row, $r_0[3] = 0$). It does not use SKY130's multi-output full-adder cell, so the count reflects the optimised logic, not the schematic.

2. Critical path. It runs from input register `a_r[1]` to output register `p[6]` (Table 4) and contains 10 cells: the launch flip-flop, 8 logic cells and the capture flip-flop. The data arrives at 2.652 ns and the setup time is 0.135 ns, giving $T_{\min} = 2.787$ ns. It crosses 7 full-adder stages, close to the $2N = 8$ estimate; one fewer because the first adder of each row has a constant 0 carry-in, so it reduces to a half adder that ABC merges with the AND gates.

3. Flip-flop share of the area. The 16 `dfrtp_1` flip-flops take $16 \times 25.024 = 400.384\,\mu m^2$ of $756.976\,\mu m^2$, or 52.89 %, although they are only 16 of 61 cells (26 %): one `dfrtp_1` is about 6.7 times a `nand2_1` ($3.754\,\mu m^2$) because it holds a master and a slave latch, clock inverters and reset circuitry. The registers grow linearly with N ($2N + 2N = 16$ bits at $N = 4$) while the array grows with N^2, so at $N = 4$ the registers dominate; at 8 or 16 bits the logic share would rise.

4. Post-route $F_{\max}$ and added cells. Post-route $F_{\max}$ is 130.92 MHz over all corners (264.72 MHz in the typical corner), against 358.81 MHz from Part C, which assumed an ideal clock and no wires. After routing, wire resistance and capacitance raise the launch flip-flop's clock-to-Q from 0.398 ns to 0.525 ns; the real clock tree adds about 0.32 ns of insertion delay with some skew, and LibreLane adds 0.25 ns of clock uncertainty; the cells are spread over a core that is only 7 % utilised, so some nets are long, and the resizer put two delay buffers (`wire19`, `max_cap18`) on the critical path (0.61 ns in the typical corner). LibreLane also re-synthesizes the RTL, so the typical-corner data arrives at 3.715 ns instead of 2.652 ns, and the all-corner figure is set by the slow corner (1.60 V, 100 °C), where the same path arrives at 7.643 ns. The flow added decap, fill, tap, port-buffer, repair-buffer and clock-buffer cells; the purpose of each kind is given in Section 4.3.

5. DRC and LVS. DRC (design rule check) verifies that the drawn geometry obeys the foundry's manufacturing rules (minimum widths and spacings, via enclosure, minimum areas, density, antenna limits); a violation means the masks may not print as drawn, giving shorts, opens or low yield. LVS (layout versus schematic) extracts the devices and connections from the layout and compares them with the netlist; a violation means the chip is not the circuit that was verified in simulation and STA. Both must pass: DRC shows the layout can be built, and LVS shows it is the right circuit. The final run has 0 Magic DRC errors, 0 KLayout DRC errors and 0 LVS errors.

6. Scaling to 8 bits. For $N = 4$ the array uses $N^2 = 16$ AND gates and $N(N-1) = 4 \times 3 = 12$ full adders. For $N = 8$ it would use $8^2 = 64$ AND gates and $8 \times 7 = 56$ full adders, about 4.7 times as many, and $2N + 2N = 32$ flip-flops instead of 16. The critical path is about $2N$ full-adder delays: $2 \times 4 = 8$ for $N = 4$ and $2 \times 8 = 16$ for $N = 8$, so the logic delay roughly doubles. From Part C, the logic part of the path is $2.652 - 0.398 = 2.254$ ns for an estimated 8 stages, about 0.28 ns per full-adder delay. For $N = 8$: $16 \times 0.28 = 4.51$ ns of logic, plus 0.398 ns clock-to-Q and 0.135 ns setup, gives $T_{\min} \approx 5.04$ ns and $F_{\max} \approx 1000/5.04 \approx 198$ MHz, compared with 358.81 MHz at $N = 4$. Area grows quadratically while delay grows only linearly.

A Commands, in order

All commands ran inside the IIC-OSIC-TOOLS container from `/foss/designs`, the repository root.

```
# host (WSL 2)
.dev/preflight.sh
docker pull hpretl/iic-osic-tools:2026.10
.dev/start_container.sh
# container: tools and PDK
cd /foss/designs
iverilog -V | head -1; yosys -V; sta -version
echo $PDK_ROOT $PDK                   # /foss/pdks ihp-sg13g2
sak-pdk sky130A                       # /foss/pdks sky130A
# Part A
iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v
vvp sim                               # PASS
#   rtl/full_adder.v: assign sum = a ^ b;          -> vvp sim: FAIL, 95 errors
#   restored assign sum = a ^ b ^ cin;             -> vvp sim: PASS
gtkwave mult.vcd
# Part B
yosys -c synth/synth.tcl
CELLS=$PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/verilog
iverilog -g2012 -DFUNCTIONAL -DUNIT_DELAY=#1 -o gls \
    tb/tb_mult.v $CELLS/primitives.v $CELLS/sky130_fd_sc_hd.v synth/mult_array_netlist.v
vvp gls                               # PASS
# Part C
sta sta/sta.tcl                          # T = 10 ns: worst slack 7.213
STA_PERIOD=2.79 sta sta/sta.tcl          # T = Tmin: worst slack 0.003
# Part D: run 1 with CLOCK_PERIOD 3.5, then +0.5 ns before each later run
#         (4.0, 4.5, 5.0, 5.5, 6.0, 6.5, 7.0, 7.5, 8.0)
cd pnr
librelane config.json
cp runs/RUN_2026-10-23_23-44-39/final/metrics.json .
cp runs/RUN_2026-10-23_23-44-39/final/gds/mult_array.gds .
klayout mult_array.gds
```

B Issues and fixes

Issue	Fix
The container's default PDK was `ihp-sg13g2`	`sak-pdk sky130A`
Mismatch messages printed picosecond values labelled "ns"	Added `$timeformat(-9, 0, " ns", 0)` to the testbench
`sta sta/sta.tcl` stayed in the interactive OpenSTA shell	Added `exit` at the end of the script
`report_worst_slack -max` printed only 2 decimals	Added `-digits 3` so $T_{\min}$ is exact to 0.001 ns